



Análisis Crítico de Sistemas Distribuidos

Caso de Estudio: Apache Kafka

Aplicaciones Distribuidas — SOFT-R — A — 7mo Nivel

Corte 1 — TA-IND-01

Autor:

Pedro Leonardo Castro López

Mayo 2026

Índice

1. Introducción	2
2. Transparencias de Distribución en Apache Kafka	2
2.1. Transparencia de Acceso	2
2.2. Transparencia de Ubicación	2
2.3. Transparencia de Migración	2
3. Modelo CAP en Apache Kafka	3
3.1. Clasificación de Kafka: Sistema AP	3
3.2. Consistencia Eventual	3
4. Modelos de Comunicación en Apache Kafka	3
4.1. Patrón Publish/Subscribe	3
4.2. Comunicación Asíncrona	4
5. Tolerancia a Fallos en Apache Kafka	4
5.1. Replicación de Particiones	4
5.2. Elección de Líder	4
5.3. Coordinación con ZooKeeper y KRaft	5
6. Conclusiones	5
Referencias	5

1. Introducción

Un sistema distribuido es un conjunto de computadoras independientes que colaboran entre sí para cumplir un objetivo común, presentándose al usuario como un único sistema unificado. Este modelo constituye la base de tecnologías modernas como la computación en la nube, servicios de streaming y aplicaciones de mensajería a gran escala.

Los sistemas distribuidos se caracterizan principalmente por su capacidad de compartir recursos y datos entre múltiples nodos, garantizando al mismo tiempo disponibilidad, escalabilidad y tolerancia a fallos. Entre los desafíos más importantes de este campo se encuentran la consistencia de los datos, la comunicación entre componentes y la coordinación ante fallos parciales del sistema.

En el presente informe se analiza **Apache Kafka** como sistema distribuido real, evaluando su diseño y funcionamiento bajo los conceptos fundamentales de la Unidad 1: transparencias de distribución, modelo CAP, modelos de comunicación y tolerancia a fallos.

2. Transparencias de Distribución en Apache Kafka

La transparencia en un sistema distribuido es la capacidad de ocultar su naturaleza distribuida a los usuarios y aplicaciones, haciendo que múltiples computadoras interconectadas parezcan un único sistema coherente. Apache Kafka implementa varios tipos de transparencia que permiten a los desarrolladores utilizarlo sin conocer su complejidad interna.

2.1. Transparencia de Acceso

Kafka oculta las diferencias entre el acceso local y remoto a los mensajes. Un productor o consumidor utiliza exactamente la misma API sin importar si los brokers están en el mismo datacenter o en servidores distribuidos geográficamente. El cliente simplemente publica o consume mensajes de un topic, sin gestionar ninguna operación de red manualmente.

2.2. Transparencia de Ubicación

Los clientes de Kafka no necesitan conocer en qué servidor físico se encuentran almacenadas las particiones de un topic. El cliente se conecta a cualquier broker del clúster y este le redirige automáticamente hacia el broker líder correspondiente. Esto permite que el sistema escale o reorganice sus nodos sin afectar a las aplicaciones que lo consumen.

2.3. Transparencia de Migración

Kafka puede reasignar particiones entre brokers de forma dinámica sin interrumpir el servicio. Por ejemplo, cuando se agrega un nuevo broker al clúster para balancear la

carga, las particiones se redistribuyen automáticamente. Los productores y consumidores continúan operando con normalidad sin requerir ningún cambio en su configuración.

3. Modelo CAP en Apache Kafka

El teorema CAP establece que un sistema distribuido no puede garantizar simultáneamente las tres propiedades siguientes: **Consistencia** (todos los nodos ven los mismos datos al mismo tiempo), **Disponibilidad** (el sistema siempre responde a las solicitudes) y **Tolerancia a particiones** (el sistema sigue funcionando aunque fallen las comunicaciones entre nodos).

3.1. Clasificación de Kafka: Sistema AP

Apache Kafka es un sistema **AP**, es decir, prioriza la **Disponibilidad** y la **Tolerancia a particiones** sobre la consistencia estricta. Kafka prefiere seguir funcionando y procesando mensajes antes que detenerse por no estar completamente disponible.

Esta decisión de diseño tiene sentido en casos de uso reales: plataformas como Netflix o LinkedIn, que utilizan Kafka, necesitan procesar millones de eventos por segundo sin interrupciones. Una caída total del sistema sería mucho más costosa que una pequeña inconsistencia temporal en los datos.

3.2. Consistencia Eventual

Kafka no garantiza consistencia inmediata entre todas las réplicas, sino **consistencia eventual**: cuando un broker falla y otro toma su lugar, puede existir un breve período en que las réplicas no estén perfectamente sincronizadas. Sin embargo, el sistema garantiza que eventualmente todos los nodos convergerán al mismo estado.

4. Modelos de Comunicación en Apache Kafka

El modelo de comunicación define cómo los componentes de un sistema distribuido intercambian información. Apache Kafka utiliza el patrón **Publish/Subscribe (Pub/Sub)**, que permite la comunicación asíncrona y desacoplada entre servicios.

4.1. Patrón Publish/Subscribe

En el modelo Pub/Sub de Kafka intervienen tres actores principales:

- **Producer (Publicador)**: componente que genera y envía mensajes a un topic de Kafka. No necesita saber quién los va a consumir.

- **Broker:** servidor de Kafka que recibe, almacena y distribuye los mensajes en particiones ordenadas.
- **Consumer (Suscriptor):** componente que lee los mensajes de un topic de forma independiente al producer.

La clave de este modelo es el **desacoplamiento**: el producer y el consumer no se conocen entre sí ni esperan el uno al otro. El producer publica el mensaje y continúa su trabajo; el consumer lo leerá cuando esté listo. Esto permite que múltiples consumidores lean el mismo mensaje de forma independiente.

4.2. Comunicación Asíncrona

A diferencia de un modelo síncrono donde el emisor espera respuesta, Kafka opera de forma completamente asíncrona. Esto lo hace ideal para sistemas de alto volumen como el registro de eventos, pipelines de datos en tiempo real y arquitecturas orientadas a eventos (event-driven).

5. Tolerancia a Fallos en Apache Kafka

La tolerancia a fallos es la capacidad de un sistema distribuido de continuar operando correctamente ante la caída parcial de sus componentes. Apache Kafka está diseñado desde su base para sobrevivir a fallos de nodos sin interrumpir el servicio.

5.1. Replicación de Particiones

Kafka almacena múltiples copias de cada partición en diferentes brokers, configurado mediante el **factor de replicación**. Con un factor de replicación de 3, cada mensaje existe en 3 nodos distintos. Si uno de esos nodos falla, los otros 2 siguen activos y continúan entregando los mensajes sin pérdida de información. Esto elimina el punto único de falla (SPOF) del sistema.

5.2. Elección de Líder

Cada partición tiene un broker designado como **líder** que maneja todas las operaciones de lectura y escritura, mientras que los demás brokers actúan como réplicas. Si el líder falla, Kafka realiza automáticamente una **elección de nuevo líder** entre las réplicas disponibles, restaurando el servicio en segundos sin intervención manual.

5.3. Coordinación con ZooKeeper y KRaft

Históricamente Kafka utilizaba **ZooKeeper** para coordinar el estado del clúster y gestionar las elecciones de líder. En versiones recientes, Kafka introdujo **KRaft**, su propio protocolo de consenso interno que elimina la dependencia de ZooKeeper, simplificando la arquitectura y mejorando la velocidad de recuperación ante fallos.

6. Conclusiones

Apache Kafka representa un ejemplo sólido de sistema distribuido moderno que implementa de forma efectiva los conceptos fundamentales estudiados en la Unidad 1.

En cuanto a las **transparencias de distribución**, Kafka oculta completamente su complejidad interna al usuario: la ubicación física de los datos, la migración de particiones y el acceso a los brokers son invisibles para las aplicaciones cliente.

Respecto al **modelo CAP**, Kafka adopta una posición AP, priorizando la disponibilidad continua del sistema sobre la consistencia inmediata. Esta decisión es coherente con sus casos de uso principales, donde la interrupción del servicio tendría un impacto mucho mayor que una breve inconsistencia temporal.

Su **modelo de comunicación** Pub/Sub permite un alto grado de desacoplamiento entre productores y consumidores, facilitando arquitecturas escalables y flexibles orientadas a eventos.

Finalmente, su mecanismo de **tolerancia a fallos** basado en replicación y elección automática de líder garantiza que el sistema continúe operando incluso ante la caída de múltiples nodos, lo que lo convierte en una herramienta confiable para entornos de producción críticos.

Referencias

1. Apache Kafka Documentation. (2024). *Kafka Official Docs*. Recuperado de: <https://kafka.apache.org/documentation/>
2. Tanenbaum, A. S., & Van Steen, M. (2017). *Distributed Systems: Principles and Paradigms*. Pearson.
3. Brewer, E. (2000). Towards robust distributed systems. *ACM Symposium on Principles of Distributed Computing (PODC)*.
4. Narkhede, N., Shapira, G., & Palino, T. (2017). *Kafka: The Definitive Guide*. O'Reilly Media.

5. Saini, M. (s.f.). *Kafka use cases and applications: Publish-Subscribe*. The Life Titbits, Medium. Recuperado el 19 de mayo de 2026, de <https://medium.com/the-life-titbits/kafka-use-cases-and-applications-ce65be1073f3>
6. Confluent. (s.f.). *Publish-Subscribe: Intro to Pub-Sub Messaging*. <https://www.confluent.io/learn/publish-subscribe/>
7. Tijero, H. P., & Gutiérrez, J. J. (2018). Desarrollo de Sistemas Distribuidos de Tiempo Real y de Criticidad Mixta a través del Estándar DDS. *Revista Iberoamericana de Automática e Informática Industrial*, 15(4), 439–447. <https://doi.org/10.4995/RIAI.2017.9000>